15.2 Index speed problems

Although we predominantly encounter search problems when users search the data, the other side of the coin is problems while indexing the data. In this section, we look at several reasons indexing operations can underperform and tips for improving indexing performance.

15.2.1 System-generated identifiers

When we use user-supplied IDs, Elasticsearch has to perform an additional step: checking whether the supplied ID already exists in the index. If the answer is no, indexing the document with that ID goes ahead. This is an unnecessary network call that takes a toll when we have thousands or millions of documents to index.

Sometimes, however, we don't need to worry about the document ID; we can let Elasticsearch create randomly generated IDs for documents. This way, a node that gets a request to index a document creates a globally unique ID and assigns it to the document instantly.

If your organization can live with random IDs, using them instead of user-defined IDs benefits performance. There's a downside, though: indexing documents without a primary key breeds duplication.

15.2.2 Bulk requests

Indexing documents using single-document APIs (indexing one by one) is asking for trouble, especially if we need to index many data documents. Fortunately, Elasticsearch provides a `_bulk` API that helps index documents in batches. (We discussed the `_bulk` API in chapter 5.) Because there isn't a batch size written in stone, my advice is to try out performance on the cluster to find an optimal size.

For example, if you need to ingest large amounts of data overnight, try increasing the refresh setting for potential benefits from the perspective of disk-usage performance. During the indexing operation, especially bulk inserts, increasing the refresh time is another tweak that most administrators use. We learn about increasing refresh times next.

15.2.3 Adjusting the refresh rate

When a document is indexed, usually it's available for search in under a second, but various operations happen under the hood. Initially, documents in the in-memory buffer are moved to segments before being stored in the filesystem cache. They then are finally flushed to disk.

All operations carried out on an index are committed once a refresh is called. If we have a large influx of documents to index, refreshing ensures that they are written to disk as well as being available for searches. We need to know whether the freshly indexed documents are available for searching instantly.